

SPC-CML — Visão geral da arquitetura

Como o sistema é montado, por onde o dado passa e o que decide o quê

Este documento descreve o SPC-CML por inteiro: as peças, a ordem em que elas atuam e a razão de cada uma existir. É autocontido e não pressupõe familiaridade com o código. Para o aprofundamento da camada de especialização gramatical por item e do contrato de artefato, ver o documento companheiro `docs/arquitetura-spc-cml.tex`.

Sumário

1	O que o sistema é, e o problema que ele ataca	3
1.1	Três domínios, uma arquitetura	3
2	Panorama: as peças e como se encaixam	3
2.1	O que cada etapa decide	5
3	A fonte única de verdade: a DSL	5
3.1	Esquema de Controle: o conhecimento do domínio	5
3.2	Esquema de Dados: o universo fechado de respostas	6
4	Derivação da gramática: $DSL \rightarrow G$	6
4.1	Dois estreitamentos deliberados	7
5	O Grafo de Conhecimento	7
6	Recuperação: da telemetria ao subgrafo	7
6.1	Metade determinística: o que é permitido	7
6.2	Metade aproximada: o que é mostrado	8
6.3	O que atravessa o fio	8
7	Especialização: $G \rightarrow \hat{G}$	8
8	Decodificação restrita	8
8.1	Grammar prompting	9
8.2	Mascaramento de logits	9
8.3	Verificação independente	9
8.4	Parâmetros de operação	9
9	O contrato do artefato e a realimentação	10
10	O caminho sem máscara: Earley e reparo	10
11	Genericidade: papéis, não nomes	11

12 Topologia de execução	11
12.1 Por que três motores	11
12.2 Interfaces	12
13 Como executar	12
13.1 Pré-requisitos	12
13.2 Subida completa	12
13.3 Inferência em lote	12
14 Avaliação	12
15 Mapa de arquivos	13
16 Onde aprofundar	14

1 O que o sistema é, e o problema que ele ataca

O SPC-CML (Compilador Semântico de Prompts) é um **middleware neuro-simbólico**. Ele se interpõe entre a fala de um profissional e a ordem que um sistema crítico executa, e faz isso com três coisas que não são um modelo de linguagem: uma **DSL formal**, um **Grafo de Conhecimento** e **decodificação restrita por gramática**.

O ponto de partida é que um LLM que traduz “a pressão tá despencando, sobe a nora e aprofunda o propofol” numa ordem de bomba de infusão erra de duas maneiras que exigem remédios diferentes.

Tipo de erro	Exemplo	Mecanismo que o elimina
Sintático	emitir dose 0.1 gotas/min, ou prosa livre em vez da linguagem de ordens	gramática BNF derivada da DSL + decodificação restrita
Semântico	aumentar Propofol com PAM 52 mmHg — sintaticamente perfeito, clinicamente perigoso	Grafo de Conhecimento avaliado sobre a telemetria, convertido em gramática

A tese operacional

O segundo tipo de erro pode ser reduzido ao primeiro. Se o grafo determina que aumentar Propofol é proibido *agora*, essa cadeia é removida da gramática e o decodificador não consegue emití-la. O erro semântico deixa de ser algo a detectar depois e passa a ser **inexprimível**.

A consequência arquitetural vale ser dita explicitamente: **a etapa que decide o que é seguro não é neural**. O LLM traduz linguagem natural em estrutura; quem decide se o Propofol pode subir é uma comparação numérica ($52 < 60$) sobre um grafo escrito por especialistas.

1.1 Três domínios, uma arquitetura

O sistema é instanciado em três domínios simultâneos, que servem para demonstrar que a maquinaria não depende de nenhum deles.

	Clínico (med)	Agrícola (agro)	Arbitragem (fut)
Situação	terapia intensiva	pulverização por drone	futebol
Modelo	med/uti.dsl	agro/lavoura.agro	fut/futebol.fut
Artefato	plano	missao	arbitragem
Cláusula	ordem	aplicacao	decisao
Item	farmaco (12)	produto (9)	infracao (10)
Meio	via	modo	reinicio
Sujeito	paciente	talhao	partida
Contexto	protocolo (3)	cultura (3)	lance (4)

Nenhum módulo do núcleo menciona “fármaco”, “produto” ou “infração”. Todos falam em **papéis** — item, decisão, meio, quantidade, sujeito, contexto — e cada domínio declara como nomeia os seus. A seção 11 detalha esse mecanismo.

2 Panorama: as peças e como se encaixam

A figura 1 mostra a arquitetura inteira. Há dois tempos sobrepostos: o *tempo de projeto*, em que a DSL é escrita e dela derivam o grafo e a gramática; e o *tempo de execução*, em que cada pedido poda a gramática antes de gerar.

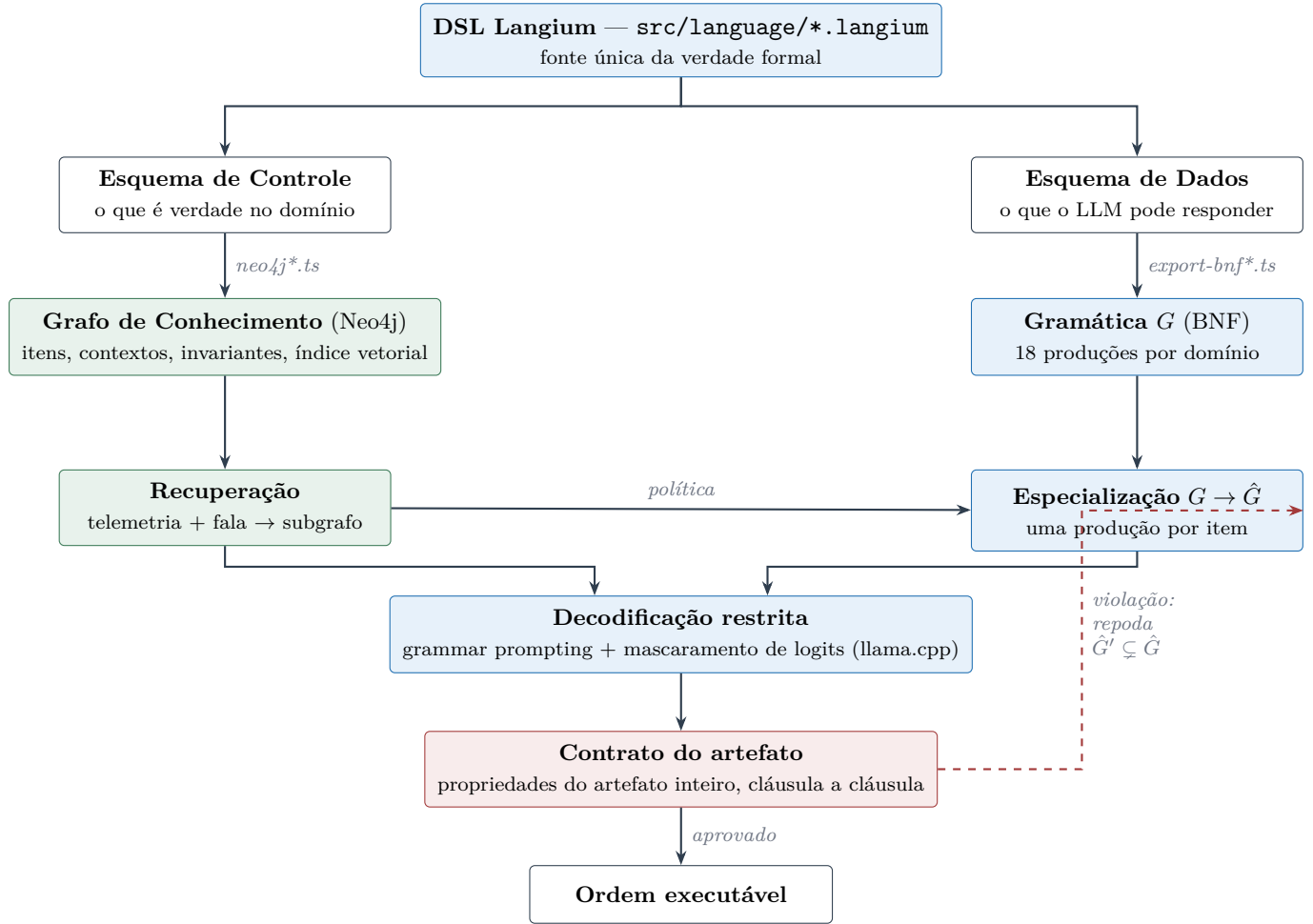


Figura 1: Arquitetura do SPC-CML. A DSL é a única fonte de verdade; dela derivam tanto o grafo quanto a gramática. Em execução, o grafo poda a gramática *antes* da geração, e o contrato realimenta a poda *depois* dela.

2.1 O que cada etapa decide

Etapa	Decide	Natureza
Recuperação no grafo	o que é permitido agora	determinística (comparação numérica)
Foco por embedding	o que é mostrado no prompt	aproximada (cosseno)
Especialização	o que é exprimível	determinística (reescrita de gramática)
Decodificação	qual cadeia admissível sai	neural, sob máscara
Contrato	se o artefato inteiro fecha	determinística (percurso de cláusulas)

A linha que não se cruza

O embedding decide apenas **o que aparece** no Prompt Semântico. Quem decide o que é **permitido** continua sendo a comparação numérica determinística. A similaridade nunca ganha autoridade sobre a segurança — e essa separação está escrita no cabeçalho de `src/knowledge/foco.ts`.

3 A fonte única de verdade: a DSL

Cada domínio tem uma gramática Langium (`src/language/`) que é particionada em dois esquemas. A partição é o que sustenta a arquitetura inteira: os dois artefatos derivados — o grafo e a BNF — saem do *mesmo* arquivo, e por isso não podem dessincronizar.

3.1 Esquema de Controle: o conhecimento do domínio

Alimenta o Grafo de Conhecimento. É escrito pelo especialista — no caso clínico, o intensivista e o farmacêutico clínico.

```
farmaco Propofol {
  classe sedativo
  rxnorm "8782" atc "N01AX10"
  alto_risco sim

  vias [ ACESSO_CENTRAL, ACESSO_PERIFERICO ]
  dose inicial 0.5 mg/kg/h
  dose maxima 4.0 mg/kg/h
  titulacao 0.5 mg/kg/h a_cada 15 min alvo "RASS -2 a 0"

  bomba { // limites DERS da bomba inteligente
    limite_leve 3.0 mg/kg/h // soft limit: alerta
    limite_rigido 4.0 mg/kg/h // hard limit: bloqueio
  }

  interacao: Midazolam gravidade moderada ("depressao respiratoria aditiva")
  contraindicado: "hipotensao refrataria com PAM < 60 mmHg"
}

// Invariante com condicao ESTRUTURADA -- avaliavel por maquina, nao por LLM
regra_seguranca: bloquear_incremento Propofol se PAM < 60.0 mmHg
  ("risco de hipotensao severa e colapso hemodinamico")
```

O detalhe decisivo está na última declaração. A invariante não é uma frase em prosa que um modelo precisa interpretar: é uma condição **estruturada** — parâmetro, operador, limiar, unidade — que uma comparação avalia. O modelo `med/uti.dsl` declara 12 fármacos, 3 protocolos, 4 populações e 6 regras de segurança.

3.2 Esquema de Dados: o universo fechado de respostas

Delimita exhaustivamente o que o LLM pode emitir. É esta seção que vira BNF.

```
esquema_dados AssistenteUTI_v2 {
  decisoes [ INICIAR_INFUSAO, AUMENTAR_VAZAO, REDUZIR_VAZAO, MANTER_BLOQUEADO,
             SUSPENDER, SOLICITAR_EXAME, ESCALAR_EQUIPE, BLOQUEAR_ORDEM ]
  vias [ ACESSO_CENTRAL, ACESSO_PERIFERICO, INTRAOSSEO, SC ]
  alertas [ INFORMATIVO, ATENCAO, CRITICO, BLOQUEANTE ]

  conduta Manter_Bloqueio {
    decisao MANTER_BLOQUEADO
    requer_dupla_checagem sim
    recurso_fhir "DetectedIssue"
  }
}

// A producao que a decodificacao restrita preenche
plano Plano_Choque_01 para Choque_Septico {
  esquema_referencia AssistenteUTI_v2
  paciente 'PT-2026-0031'
  sequencia [ Manter_Bloqueio, Acionar_Equipe ]
  ordem Propofol decisao MANTER_BLOQUEADO dose 0.0 mg/kg/h via ACESSO_CENTRAL
    justificativa 'PAM 52 mmHg abaixo de 60'
  alerta CRITICO 'hipoperfusao grave' regra 'Choque_Septico/escalonar'
  auditoria 'plano derivado sob restricao gramatical'
}
```

As condutas são declaradas **dentro** de um `esquema_dados`, e um `plano` só pode citar condutas do esquema que ele próprio referencia. Isso é imposto por um `ScopeProvider` dedicado (`src/language/dsl-module.ts`), de modo que o universo é fechado também na resolução de nomes, não apenas na sintaxe.

4 Derivação da gramática: $DSL \rightarrow G$

Antes, a BNF era mantida à mão em paralelo à DSL. Duas fontes de verdade para a mesma linguagem significam que qualquer evolução do domínio pode dessincronizar o decodificador do modelo — justamente onde a segurança é decidida.

Hoje `src/cli/export-bnf.ts` **deriva** a BNF de duas fontes locais já validadas:

Fonte	O que fornece
<code>src/language/dsl.langium</code>	a estrutura da saída (a regra <code>PlanCommand</code> e o que ela alcança) e os vocabulários fechados (<code>Decision</code> , <code>Route</code> , <code>AlertLevel</code> , <code>Unit</code>), lidos da AST da gramática <code>Langium</code>
<code>src/examples/med/uti.dsl</code>	as instâncias permitidas — nomes de fármacos, condutas, protocolos — lidas do modelo do especialista

O resultado é `src/python_engine/grammar/advanced_icu.bnf`, com 18 produções:

```
plano ::= "plano" identificador "para" protocolo "{ "esquema_referencia" esquema
  "paciente" texto "sequencia" "[" conduta plano_g1* "]"
  plano_g2* plano_g3* "auditoria" texto "}"
ordem ::= "ordem" farmaco "decisao" decisao "dose" quantidade "via" via
  "justificativa" texto
farmaco ::= "Noradrenalina" | "Adrenalina" | "Vasopressina" | "Dobutamina"
  | "Propofol" | "Midazolam" | "Fentanil" | "Cisatracurio"
  | "Vancomicina" | "Piperacilina_Tazobactam" | "Heparina"
  | "Insulina_Regular"
unidade ::= "mcg/kg/min" | "U/min" | "mg/kg/h" | "mcg/kg/h" | "mg/kg" | "mg"
  | "UI/kg" | "UI/h"
```

Os 12 fármacos vieram do `uti.dsl`. Citar `Dopamina` deixa de ser um erro semântico a detectar depois: **não existe na gramática**.

4.1 Dois estreitamentos deliberados

A gramática de saída é *mais restrita* que a DSL de autoria, por decisão de projeto — ou seja, $L(G_{\text{saída}}) \subsetneq L(\text{DSL})$:

- **justificativa** e **regra** passam de opcionais a **obrigatórias**: toda ordem gerada automaticamente carrega rastreabilidade;
- **texto** aceita apenas aspas simples, porque o leitor BNF do motor Python trata `|` como separador de alternativas.

Todo plano gerado é um programa válido da DSL; a recíproca não vale.

5 O Grafo de Conhecimento

`src/database/neo4j.ts` percorre a AST do modelo e mapeia o Esquema de Controle para o Neo4j: um nó por fármaco, protocolo, população e invariante, com as arestas que os ligam. Junto com os nós, sincroniza um **índice vetorial**: o documento textual de cada nó é montado em `src/knowledge/documentos.ts` a partir da gramática do próprio domínio, e o vetor vem do `endpoint /embed` do motor Python.

O banco roda como serviço Docker (`neo4j:5.15.0`), exposto em 7474 (HTTP) e 7687 (Bolt).

Por que a sincronização vem depois do aquecimento

Cada nó embutido custa uma chamada a `/embed`. Com o motor frio, a primeira dessas chamadas esperaria a carga inteira do modelo. Por isso o `init.sh` aquece os motores (passo 5) *antes* de sincronizar os grafos (passo 6), e não o contrário.

6 Recuperação: da telemetria ao subgrafo

Esta é a etapa em que o sistema decide o que é permitido. Ela tem duas metades com naturezas diferentes, deliberadamente não misturadas.

6.1 Metade determinística: o que é permitido

`src/knowledge/graphrag.ts` avalia a telemetria contra o modelo e produz uma **política por fármaco**.

```
PAM=52 FC=145 lactato=4.8 RASS=2 TFG=28 plaquetas=45 glicemia=210
|
v
Protocolos ativados: Choque_Septico (lactato 4.8 > 2), Sedacao_VM (RASS 2 > 0),
                    Controle_Glicemico_UTI (glicemia 210 > 180)
Bloqueios: Propofol (PAM 52 < 60), Noradrenalina (FC 145 > 130),
           Cisatracurio (RASS 2 > -4), Heparina (plaquetas 45 < 50)
Vetos: Dobutamina (protocolo), Midazolam (protocolo)
Ajustes: Vancomicina x0.5, Fentanil x0.75, Midazolam x0.5 (TFG 28)
Escalonamentos: TIME_RESPOSTA_RAPIDA, INTENSIVISTA
```

Uma regra de decisão importante: invariantes cujo parâmetro **não está presente** na telemetria são ignoradas. Ausência de medida não é evidência de normalidade, mas também não autoriza bloquear.

6.2 Metade aproximada: o que é mostrado

`src/knowledge/foco.ts` decide, a partir do texto digitado, quais nós o pedido alcança — por similaridade de cosseno contra o índice vetorial — e quantos deles sobrevivem, por um corte *relativo* ao melhor candidato. Quando o pedido não nomeia nada, o módulo escolhe de que lado partir pelo índice que se compromete mais com a própria resposta.

Esse módulo é agnóstico de domínio por construção: não carrega lista de palavras de domínio nenhum nem constante calibrada para um modelo específico. Tudo o que sabe, lê do modelo carregado e do índice vetorial.

6.3 O que atravessa o fio

O resultado viaja para o motor Python em três partes (`src/knowledge/politica.ts`):

<code>politicas</code>	por item: decisões, meios, unidades e valores admissíveis (o reticulado de dose derivado do próprio modelo)
<code>papeis</code>	como este domínio nomeia item, decisão, meio, quantidade, sujeito e contexto
<code>constantes</code>	sujeito e contextos em foco — são <i>dado</i> , não decisão, e por isso viram literal na gramática, não campo a preencher

7 Especialização: $G \rightarrow \hat{G}$

`src/python_engine/grammar_from_kg.py` converte a política em gramática. O ganho central é a especialização **por item**: em vez de podar três vocabulários independentes, o motor emite **uma produção por item**.

```
ordem ::=ordem_noradrenalina | ordem_propofol | ordem_vasopressina | ...

ordem_propofol ::= "ordem" "Propofol" "decisao" decisao_propofol
                  "dose" quantidade_propofol "via" via_propofol
                  "justificativa" texto
decisao_propofol ::= "REDUZIR_VAZAO" | "MANTER_VAZAO" | "MANTER_BLOQUEADO"
                  | "SUSPENDER" | "SUBSTITUIR" | "SOLICITAR_EXAME"
                  | "ESCALAR_EQUIPE" | "BLOQUEAR_ORDEM"
unidade_propofol ::= "mg/kg/h"
via_propofol ::= "ACESSO_CENTRAL" | "ACESSO_PERIFERICO"
```

AUMENTAR_VAZAO desapareceu de `decisao_propofol`. Note também que `unidade_propofol` admite só `mg/kg/h` — trocar a unidade de um vasopressor pela de um sedativo, erro clássico de medicação, também se torna inexprimível.

Por que “por item” é a diferença material

Com uma lista global de decisões, `Propofol` + `AUMENTAR_VAZAO` continuaria gramatical, porque *algum* fármaco admite aumentar. A regra `ordem` é um produto cartesiano; achatar nela a política por item perde exatamente a informação que decidia algo.

Este é o ponto em que o documento companheiro `docs/arquitetura-spc-cml.tex` entra em detalhe: o algoritmo de especialização, a origem dos valores e o efeito medido.

8 Decodificação restrita

Com $\hat{G}$ em mãos, o motor monta o prompt e gera sob máscara.

8.1 Grammar prompting

`prompt_builder.py` monta o prompt no formato $(x_i, G[y_i], y_i)$ para cada exemplar *few-shot*, seguido do x de teste. O Prompt Semântico vindo do grafo entra como bloco de contexto factual.

Há uma diferença deliberada em relação a Wang et al. (2023). No grammar prompting original, o LLM primeiro *prediz* $G[y]$ e só então gera y . No SPC-CML essa etapa não cabe ao modelo: $\hat{G}$ vem da poda no grafo — e essa substituição é a contribuição da arquitetura. As regras são **dadas**, não pedidas.

8.2 Mascaramento de logits

O `bnf.py` compila $\hat{G}$ para GBNF, e o `llama.cpp` mascara os logits em C++. A gramática é compilada *uma vez* por requisição.

Backend	Variável	Característica
llamacpp	padrão	GBNF compilada uma vez, mascaramento em C++
outlines	SPC_CML_BACKEND	CFG incremental em Python; reconstrói um FSM sobre todo o vocabulário a cada terminal, o que nesta gramática não termina em tempo útil. Mantido para comparação de desempenho

8.3 Verificação independente

Mesmo com mascaramento, a saída é **reparseada** contra $\hat{G}$ antes de sair do serviço (`parser_hat.parse`, em `main.py`). A resposta carrega `valido`, `regras_em_g_hat` e uma marca **especializada** que diz se a especialização por item chegou a rodar — sem ela, um processo antigo ainda no ar podaria só pelos três vocabulários e devolveria um plano com aparência normal.

8.4 Parâmetros de operação

Variável	Padrão	Razão
SPC_CML_N_CTX	8192	prompt completo + exemplares ≈ 3800 <i>tokens</i> ; 4096 estourava
SPC_CML_MAX_TOKENS	1024	512 truncava o plano no meio
SPC_CML_MAX_ITENS	4	teto por lista; sem ele o modelo repete a mesma ordem
SPC_CML_REPEAT_PENALTY	1,15	a máscara não impede repetir item admissível
SPC_CML_TEMPERATURA	0,3	—
SPC_CML_N_GPU_LAYERS	−1	todas as camadas na GPU; ignorado em <i>build</i> só-CPU

O modelo padrão é o Qwen2.5-7B-Instruct em GGUF Q4_K_M ($\approx 4,7$ GB), e o *embedder* é o bge-m3 Q8_0, que roda em CPU de propósito — a VRAM fica inteira para o modelo de geração.

Ordem de carga que não é arbitrária

O `get_embedder()` carrega o LLM principal *primeiro*, de propósito. O `llama.cpp` inicializa um contexto CUDA mesmo para `n_gpu_layers=0`, e reservar esse custo antes dos 4,7 GB de pesos faz o `cudaMalloc` falhar por pouco. Um `RLock` serializa toda chamada real ao `llama.cpp`: o *binding* não é seguro para uso concorrente sobre o mesmo dispositivo.

9 O contrato do artefato e a realimentação

A especialização por item torna inexprimível tudo o que depende de *uma* cláusula isolada. O que sobra são propriedades do artefato **inteiro**, e essas nenhuma gramática livre de contexto expressa sem explodir:

- o mesmo item aparecer duas vezes com decisões que se cancelam (INICIAR_INFUSAO seguido de AUMENTAR_VAZAO para o mesmo fármaco);
- a **sequencia** declarada no cabeçalho não corresponder às cláusulas emitidas;
- um escalonamento que o grafo disparou não virar cláusula nenhuma;
- o identificador do sujeito não ser o do cenário.

O `src/knowledge/contrato.ts` verifica essas propriedades **cláusula a cláusula**, na ordem em que foram geradas. A primeira violação devolve um **reparo**: o par (item, decisão) que deve sair da política antes da próxima tentativa.

A propriedade que fecha o ciclo

A gramática da tentativa seguinte é **estritamente menor** que a da anterior. O mesmo erro deixa de ser esprimível — a verificação semântica *realimenta a poda*, em vez de só reprovar no fim.

10 O caminho sem máscara: Earley e reparo

O mascaramento de logits só é possível com modelo local. Nenhum provedor de API (Claude, GPT, Gemini) expõe a máscara, e a arquitetura é explícita em ser agnóstica de provedor. Para esse caminho vale a estratégia do Algoritmo 1 de Wang et al. (2023): decodificar livremente, parsear, e em caso de falha extrair o maior prefixo válido.

O `src/grammar/earley.ts` implementa um reconhecedor de Earley **sem scanner** — sobre caracteres, não sobre *tokens* — que fornece as três quantidades necessárias: se $y \in L(\hat{G})$, o maior prefixo válido y_{prefix} , e o conjunto $\Sigma[y_{\text{prefix}}]$ de terminais que podem continuar.

A ausência de *scanner* é deliberada: um *lexer* separado decidiria, por exemplo, que **Noradrenalina** é palavra reservada antes de o *parser* dizer se ali cabia um identificador livre, e a análise de prefixo herdaria esse erro.

```
ordem Noradrenalina decisao AUMENTAR_VAZAO dose 0.1 mcg/kg/min ...
      ^
      rejeitado exatamente aqui (offset 173 de 512)

Sigma[y_prefix] = "REDUZIR_VAZAO" | "MANTER_VAZAO" | "MANTER_BLOQUEADO"
                  | "SUSPENDER" | "SUBSTITUIR" | "SOLICITAR_EXAME"
                  | "ESCALAR_EQUIPE" | "BLOQUEAR_ORDEM"
```

O **reparo determinístico** completa o programa escolhendo, a cada passo, o terminal que minimiza o número de terminais ainda necessários para fechar a derivação inteira — incluindo o fechamento de toda a pilha de produções pendentes.

Por que o critério precisa ser global

Um critério guloso local nunca fecha uma lista. Diante de **sequencia** [**Manter_Bloqueio**, o símbolo **,** pertence a uma produção curta e parece mais barato que **]**, e o reparo repetiria conduta indefinidamente.

Por construção o reparo termina e o resultado é aceito por $\hat{G}$. É a rede de segurança que sustenta a garantia mesmo quando o LLM nunca converge.

11 Genericidade: papéis, não nomes

O núcleo nunca menciona um nome de domínio. Cada `graphrag-*.ts` declara os papéis do seu:

```
export const PAPEIS_MED: PapeisDominio = {
  artefato: 'plano', clausula: 'ordem', item: 'farmaco',
  decisao: 'decisao', meio: 'via', quantidade: 'quantidade',
  campoQuantidade: 'dose', campoSujeito: 'paciente', contexto: 'protocolo',
  decisoesDeIncremento: ['INICIAR_INFUSAO', 'AUMENTAR_VAZAO', 'AJUSTAR_DOSE'],
  decisaoDeEscalonamento: 'ESCALAR_EQUIPE'
};

export const PAPEIS_AGR0: PapeisDominio = {
  artefato: 'missao', clausula: 'aplicacao', item: 'produto',
  decisao: 'decisao', meio: 'modo', quantidade: 'quantidade',
  campoQuantidade: 'vazao', campoSujeito: 'talhao', contexto: 'cultura',
  decisoesDeIncremento: ['INICIAR_APLICACAO', 'AUMENTAR_VAZAO'],
  decisaoDeEscalonamento: 'ACIONAR_AGRONOMO'
};
```

Do lado Python, o `grammar_from_kg.py` mantém um `MAPA_PODA` que liga o nome da regra na BNF ao papel correspondente no subgrafo. Os três domínios convivem no mesmo mapa, porque uma BNF só traz os nomes do seu próprio domínio.

Acrescentar um quarto domínio exige: escrever a `.langium`, escrever o modelo de instâncias, declarar os papéis e acrescentar as entradas do `MAPA_PODA`. Nada no núcleo — foco, contrato, especialização, Earley — precisa ser tocado.

12 Topologia de execução

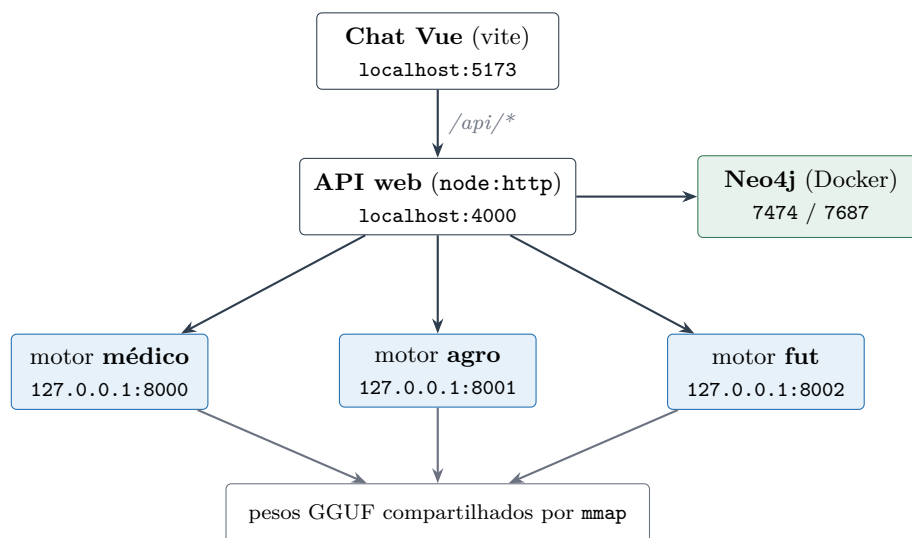


Figura 2: Processos em execução local. São três motores porque o `main.py` fixa a gramática na importação do módulo.

12.1 Por que três motores

O `main.py` lê `SPC_CML_DOMINIO` na importação e fixa ali a gramática, os exemplares e a persona do prompt. Os três domínios que o chat oferece lado a lado não cabem num processo só.

O custo de RAM não triplica: os pesos são os *mesmos arquivos* GGUF nos três processos, e o `llama.cpp` os mapeia com `mmap` — as páginas do modelo são compartilhadas pelo cache do sistema operacional.

12.2 Interfaces

Serviço	Rota	Função
Motor Python	GET /health	estado, domínio, regras em G , se o modelo está carregado
	GET /grammar	a gramática G em uso
	POST /verify	$y \in L(G)$?
	POST /embed	vetor de um texto (índice do Neo4j)
	POST /generate-constrained	geração sob $\hat{G}$
	POST /generate-fragment	decodificação incremental, um elemento por vez
API web	POST /validar-plano	juízo semântico pela LLM (avaliação)
	GET /api/dominios	domínios disponíveis
	POST /api/comando	sorteio de cenário + grafo + foco + geração
	GET POST PUT /api/chats	persistência das conversas
	POST /api/avaliar	avaliação de um lote contra o grafo

13 Como executar

13.1 Pré-requisitos

Node.js 18+, Docker (para o Neo4j) e **Python 3.10 a 3.12**.

O teto do Python é real

O `outlines 0.0.46` exige `numpy<2.0.0`, e o último `numpy 1.x (1.26.4)` só publica `wheel` até `cp312`. Em Python 3.13+ o `pip install` falha na resolução. Crie o ambiente com `py -3.12 -m venv venv`.

No Windows, o `llama-cpp-python` não tem `wheel win_amd64` no PyPI — use o índice pré-compilado (`--extra-index-url`, ver `init.sh`). Sem isso o `pip` tenta compilar o `sdist` e estoura o limite de 260 caracteres de caminho do Windows.

13.2 Subida completa

```
docker compose up -d neo4j # so o Neo4j; 'up' sem argumento reserva GPU
bash src/scripts/local/init.sh # sobe tudo e deixa os modelos quentes
```

O `init.sh` executa sete passos: pré-requisitos, Neo4j, derivação das três BNF, subida dos três motores, aquecimento dos modelos, sincronização dos grafos e, por fim, API web e chat. Variantes: `--sem-sync` (subida mais rápida), `--sem-chat` (só infraestrutura), `--parar`.

13.3 Inferência em lote

Depois do `init.sh`, os scripts de `deploy` encontram o motor do domínio certo já no ar e vão direto para a inferência:

```
bash src/scripts/local/deploy-med.sh # ou deploy-agro.sh / deploy-fut.sh
```

Cada um regenera os artefatos do Langium, compila o TypeScript, deriva a BNF do modelo, ancora o modelo no Neo4j e roda o lote de cenários.

14 Avaliação

Há dois avaliadores, com finalidades distintas.

O `src/inference/avaliar.ts` compara contra um **gabarito fixo** e produz três métricas: sintaxe correta, semântica correta (toda ordem esperada aparece?) e violações (há incremento

para item que o grafo bloqueou?). Quando o registro já traz um **valido** booleano — caso da arquitetura, que reparseia contra $\hat{G}$ — esse valor é usado diretamente, por ser a verificação mais autoritativa que existe. Só cai para a checagem estrutural leve quando **valido** não vem, caso esperado do *baseline*, que não passa pela decodificação restrita.

O `src/inference/avaliar-grafo.ts` não usa resposta canônica pré-computada. Cada registro carrega o sorteio completo do cenário, o que permite refazer a recuperação no grafo **linha a linha** no momento da avaliação. São duas fontes de veredito, deliberadamente não misturadas:

Violação de segurança	determinística — compara as ordens do plano contra os bloqueios e vetos que o grafo retornou para aquele cenário
Correção semântica ampla	julgada pela LLM local, via POST <code>/validar-plano</code>

Os lotes de referência são `cenarios-200-med.jsonl`, `cenarios-200-agro.jsonl` e `cenarios-200-fut.jsonl` — 200 cenários por domínio.

15 Mapa de arquivos

```
src/
  language/ dsl.langium, agrodrono.langium, futebol.langium
             *-module.ts -- servicos Langium + ScopeProvider
generated/ gerado por 'langium generate' (nao editar)
examples/
  med/ agro/ fut/ modelo do especialista + cenarios + gabarito

grammar/ -- CAMADA GRAMATICAL --
  extract-bnf.ts AST Langium + modelo -> BNF (G)
  vocabularies*.ts instancias permitidas por dominio
  constrain.ts G + politica do grafo -> G_hat (por item)
  earley.ts reconhecedor sem scanner, Sigma[y_prefix] e reparo

knowledge/
  graphrag*.ts telemetria -> subgrafo de restricoes (um por dominio)
  foco.ts recuperacao por similaridade (agnostico de dominio)
  documentos.ts documento textual de cada no, para o indice vetorial
  politica.ts papeis e politica por item -- o que atravessa o fio
  contrato.ts propriedades do artefato inteiro

database/ neo4j.ts, neo4j-agro.ts, neo4j-fut.ts (AST -> grafo)

cli/ validate.ts, export-bnf*.ts, pipeline.ts,
      inspecionar-foco.ts

inference/
  llm-client.ts Prompt Semantico + chamada ao motor
  decodificacao.ts decodificacao sob contrato, com repoda
  batch-client.ts bateria de cenarios
  avaliar*.ts metricas contra gabarito / contra o grafo

python_engine/ -- MOTOR DE DECODIFICACAO --
  bnf.py BNF -> Lark / GBNF; G[y] especializada
  grammar_from_kg.py poda de G pelo subgrafo -> G_hat
  prompt_builder.py prompt few-shot (x, G[y], y)
  main.py API FastAPI
  grammar/*.bnf GERADO -- nao editar a mao
  data/*.jsonl exemplares few-shot (3 por dominio)

web/server.ts ponte HTTP entre o chat Vue e a logica do CLI
scripts/ init.sh, deploy-*.sh (local e docker)

web-chat/ front-end Vue (projeto npm separado)
docs/ este documento e arquitetura-spc-cml.tex
```

16 Onde aprofundar

Assunto	Onde
Especialização por item, contrato de artefato, decodificação incremental	<code>docs/arquitetura-spc-cml.tex</code>
Instalação passo a passo, extensão do modelo clínico, limitações conhecidas	<code>README.md</code>
Razão de cada parâmetro e de cada ordem de carga	comentários em <code>src/python_engine/main.py</code>
Separação entre o que é mostrado e o que é permitido	cabeçalho de <code>src/knowledge/foco.ts</code>

Em uma frase

A DSL é a única fonte de verdade; dela saem o grafo e a gramática. O grafo, avaliado sobre a telemetria por comparação numérica, poda a gramática antes da geração; o contrato, percorrendo o artefato cláusula a cláusula, repoda depois dela. O modelo de linguagem opera sempre dentro do que restou — e o que ele não pode dizer, ele não diz.